Smart Agricultural Management System

Object Oriented Analysis and Software Engineering (OOASE) – Project Report

Submitted by: **Sanofar Fathima I**

1. Introduction

1.1 Overview

The Smart Agricultural Management System is a web-based agrarian IoT platform that lets agriculturalists, commercial farmers, and regional administrators monitor real-time soil telemetry, automate precision micro-drip irrigation, inspect reservoir capacities, and manage regional crop health from one central place. Soil moisture, chemical reaction (pH), primary macronutrients (Nitrogen, Phosphorus, Potassium), and temperature must be accurate, robust, and accessible across online and offline field settings, so the system uses role-based access for three primary users: platform administrators, registered farmers, and public agricultural customers.

The system follows a decoupled client-server architecture: a responsive single-page application built with modern vanilla ES6+ JavaScript and styled with an Obsidian Dark aesthetic talks to a Node.js and Express REST API, and all persistent operational data is stored in a structured JSON database. The report documents the project using OOASE practices: requirements prepared in MS Word, UML models, implementation in VS Code, Node.js and Express backend, API testing with Postman, and project and issue tracking in Jira.

1.2 Problem Statement

Many Tamil Nadu farmers still manage irrigation, fertilizer applications, and soil diagnostics through manual inspection and uncoordinated schedules, which causes recurring problems:

- **Scattered soil data** – moisture, pH, and nutrient records are scattered across paper lab notes and quickly become outdated.
- **Manual irrigation control** – pump sets and field valves are operated manually on-site, leading to irregular watering and wasted electricity.
- **Over-irrigation and water stress** – crops receive either excess water causing root hypoxia or deficient water during sensitive reproductive phases because soil moisture is not measured in real time.
- **Delayed nutrient diagnostics** – chemical soil lab reports take weeks to arrive, missing the window for corrective top-dressing fertilization.
- **Unmonitored reservoir levels** – regional farmers lack visibility into state dam storage capacities (Mettur, Bhavani, Vaigai) during critical cropping seasons.
- **Poor reporting** – irrigation histories, electricity consumption, and crop advisory logs are compiled by hand when needed.

1.3 Objectives

- Maintain farmland telemetry, soil diagnostics, irrigation logs, and crop guidelines in one place.
- Let farmers view live volumetric soil moisture, pH, NPK levels, and soil temperature with automatic fluctuation tracking.
- Let farmers remotely toggle micro-drip solenoid valves with instant reactive feedback and automated moisture accumulation.
- Let administrators calibrate sensor node fleets across 38 Tamil Nadu districts, audit system uptime, and monitor revenue analytics.
- Guarantee multi-protocol operational resilience with dual-mode operation for both online networked servers and offline file environments.
- Secure the system with login, role-based access, and model it with standard UML diagrams.

1.4 Scope of the Project

The system covers the complete agricultural monitoring cycle: the farmer registers an account and selects farmlands, the IoT node grid reports soil moisture and telemetry, the automated system triggers micro-drip solenoid valves when water stress occurs, the farmer inspects real-time crop health advisories, and the administrator reviews platform telemetry and system performance.

- **Included:** user registration and authentication, role management (Farmer, Admin, Customer), live telemetry dashboard (Moisture, pH, NPK, Temperature), solenoid valve actuation toggle, dam storage radar gauges (Mettur, Bhavanisagar, Vaigai, Aliyar), customer directory, farmer testimonials, creator admin nexus, and offline localStorage fallback.
- **Not included:** physical hardware sensor fabrication (simulated 12-second random-walk IoT telemetry stream implemented), satellite radar imagery processing, and native mobile compilation (responsive web client provided; see Section 6.3).

1.5 Technologies Used

Every layer uses JavaScript, which keeps the code base consistent across client and server. A structured JSON persistence store suits the project because farmland records, telemetry readings, and farmer logs are naturally stored as self-contained documents.

Layer	Technology	Purpose
Frontend	Vanilla ES6+ JavaScript, Semantic HTML5	Single-page responsive UI, reactive DOM binding, client routing
Styling and UI	Vanilla CSS3 (Design Tokens)	Obsidian Black, Stark White, Cobalt Blue, and Electric Cyan styling
Typography	Google Fonts (Plus Jakarta Sans, JetBrains Mono)	Clean modern interface typography and monospace telemetry metrics
Backend	Node.js (v24 LTS), Express.js 5	REST API, business logic, sensor fluctuation loop, routing
Database	Flat-File JSON Data Store (db.json)	Document storage with synchronous file system persistence
Resilience	Dual-Mode Resilient Proxy Pattern	Seamless failover between REST API server and local browser cache
Requirements	MS Word	Requirement analysis and specification document
Modelling	UML (use case, class, sequence, activity, state chart, component, deployment)	Object-oriented analysis and software design diagrams
Testing	Postman, Node.js Assertion Suite	Endpoint assertion, RBAC gate tests, regression test scripts
Tracking	Jira	Backlog management, sprint planning, and issue tracking
Tooling	VS Code, npm, Git, GitHub	Source editing, package management, distributed version control

1.6 Organisation of the Report

Section 2 presents the requirement analysis; Section 3 the seven UML diagrams; Section 4 the project code, architecture, testing and Jira tracking; Section 5 the screens of the website; and Section 6 the conclusion, limitations, future enhancements and references.

2. Requirement Analysis

2.1 Approach

Requirements were analysed in the object-oriented style: actors were identified first, their goals written as use cases, the real-world agricultural entities they handle identified as classes, and the connecting rules captured as business rules. The requirements were refined by walking through the normal agronomic workflow: the sensor node grid reports live soil physics, the server evaluates moisture thresholds, the farmer inspects crop diagnostics and toggles irrigation, and the administrator audits system telemetry.

2.2 User Roles

Role	Description	Main responsibilities
Administrator	Platform creator and lead architect with full access.	Calibrates fleet sensor nodes across all 38 districts, audits system uptime, monitors financial revenue, views real-time audit logs.
Farmer	Registered agricultural grower.	Inspects personal land telemetry, toggles micro-drip solenoid valves, updates farm profile and acreage, reviews dam storage radar.
Customer	Agricultural buyer or agro-client.	Browses public farm showcases, reviews soil fertility guidelines, submits field reviews and testimonials.
Visitor	Unregistered person.	Browses public overview and crop guides; must sign in to actuate irrigation or view personalized telemetry.

2.3 Functional Requirements

ID	Requirement	Module
FR-01	Register a farmer account with email, password, district, and farm acreage; enforce mandatory terms acceptance.	Authentication
FR-02	Authenticate users and restrict access by role (Farmer, Admin, Customer); reject unauthorized credentials.	Authentication
FR-03	Provide an isolated Creator Admin Command Gate strictly reserved for Sanofar Fathima I; block non-admin accounts.	Admin Gate
FR-04	Display farmland cards with district, locality, soil type, current crop, live moisture %, pH, and NPK ratio.	Telemetry
FR-05	Simulate realistic 12-second diurnal sensor fluctuations for moisture (+/-0.3%) and temperature (+/-0.2%).	Sensor Simulation
FR-06	Let a farmer toggle micro-drip solenoid valves between ACTIVE (45 L/hr flow) and CLOSED states.	Irrigation
FR-07	Automatically increase soil moisture (+1.2% delta) when an irrigation valve is actively dispensing water.	Irrigation
FR-08	Filter farmlands dynamically across 10 agro-climatic zones of Tamil Nadu (Cauvery Delta, Western, Southern, Hills).	Farmlands
FR-09	Display live dam storage radar gauges for Mettur Stanley, Bhavanisagar, Vaigai, and Aliyar reservoirs.	Reservoir Radar
FR-10	Show crop health score (0-100), risk level (Low, Moderate, High), and actionable agronomic AI advisory.	AI Diagnostics
FR-11	Allow farmers to edit personal profile details, farm sizing, primary crops, and upload display pictures.	Farmer Profile
FR-12	Maintain a verified farmer customer registry displaying acreage, crops, and verification status badges.	Customer Directory

ID	Requirement	Module
FR-13	Display commercial SaaS revenue figures (Total Revenue, MRR, Acreage Managed, Water Conserved).	Revenue
FR-14	Render subscription pricing tiers (Uzhavar Starter, Kaveri Precision Pro, Agro-Estate Enterprise).	Revenue
FR-15	Allow verified growers to submit reviews and field testimonials with rating scores.	Reviews
FR-16	Enable system-wide sensor recalibration across all 38 districts via an administrative override.	Admin Nexus
FR-17	Maintain real-time administrative telemetry audit logs with timestamps and node identifiers.	Admin Nexus
FR-18	Provide an agricultural knowledge hub with operating guides, fertigation tables, and crop calendars.	Agri-Tech Guide
FR-19	Display real-time commodity spot prices for major Tamil Nadu crops (Paddy, Coconut, Turmeric).	Commodity Ticker
FR-20	Automatically switch to client-side localStorage offline cache if the backend REST server is unreachable.	Resilience
FR-21	Provide a public system health-check and metadata endpoint (/api/system).	Operations
FR-22	Expose RESTful endpoints for CRUD operations on land reports, user accounts, and reviews.	API Gateway
FR-23	Persist all database updates atomically to backend/data/db.json.	Persistence

2.4 Non-Functional Requirements

Attribute	Requirement
Security	Passwords stored with secure validation; strict RBAC checkpoint blocks unauthorized access to the Creator Admin Nexus; CORS origin control and JSON body size limits.
Performance	REST API endpoints respond in under 30 ms under typical loads; lightweight vanilla client loads in under 1.2 seconds without external bundle compilation overhead.
Usability	High-contrast Obsidian Dark aesthetic adhering to WCAG 2.1 AA standards for outdoor sunlight readability; responsive layouts for desktop, tablet, and field kiosks.
Reliability	99.98% operational uptime; dual-mode proxy pattern ensures seamless fallback to local in-memory seed data if network connectivity drops.
Maintainability	Clean separation of concerns: modular server routes, controllers, and flat-file database helper; client code separated into api.js and app.js controllers.
Portability	Runs in any modern evergreen browser (Edge, Chrome, Firefox, Safari); backend runs on standard Node.js LTS runtimes across Windows, Linux, and macOS.
Auditability	Administrative sensor recalibrations, valve actuations, and user logins are logged with UTC timestamps and node IDs for regulatory traceability.

2.5 Hardware and Software Requirements

Item	Minimum requirement
Processor, memory, storage	Dual-core 1.5 GHz or higher, 2 GB RAM (4 GB recommended for development), 1 GB free disk space.
Operating system / runtime	Windows 10/11, Ubuntu 20.04 or later, or macOS; Node.js v20 or v24 LTS and npm.
Database / browser	Flat-File JSON database engine (db.json); current Microsoft Edge, Google Chrome, Mozilla Firefox, or Safari.

Item	Minimum requirement
Tools	VS Code, Postman, MS Word, ArgoUML / StarUML, Git, Jira Software account.

2.6 Data Requirements

Entity	Principal attributes	Description
User	id, name, email, password, role, phone, district, avatar, status	A person who can log in; the role decides their privilege level.
LandReport	id, district, zone, locality, coordinates, soilType, currentCrop, telemetry, irrigationStatus, aiDiagnostic	An agricultural land unit representing an agro-climatic zone.
Telemetry	soilMoisture, ph, nitrogen, phosphorus, potassium, temperature, humidity	Real-time IoT physical and chemical sensor parameters.
IrrigationStatus	automatedValve, lastIrrigated, nextCycle, powerSource	Current state and operational schedule of micro-drip valves.
AIDiagnostic	healthScore, status, advisory, riskLevel	Automated agronomic diagnostic rating and recommendations.
Revenue	totalRevenueINR, mrrINR, acresMonitored, waterSavedLiters, streams, plans	Commercial performance metrics and subscription tiers.
Review	id, author, location, crop, rating, date, title, comment, verifiedFarmer	Community testimonial and field evaluation from growers.
SystemMetadata	name, region, creator, version, lastUpdated	Global platform identity, version token, and architectural credits.

2.7 Business Rules

- **Creator Exclusivity:** The Creator Admin Command Nexus is strictly reserved for Sanofar Fathima I. All non-admin attempts to access the portal are rejected with an administrative block modal.
- **Mandatory Terms Acceptance:** A new farmer account can only be registered if the terms and conditions checkbox is explicitly acknowledged.
- **Actuation Sensory Feedback:** When a micro-drip valve is toggled to ACTIVE, the system immediately applies a +1.2% moisture increase and updates the last irrigated timestamp.
- **Automated Solenoid Flow Rate:** Active micro-drip irrigation dispenses precision water at an engineered baseline of 45 Liters per hour.
- **Critical Moisture Threshold:** Soil moisture below 40% triggers a water stress alert; automatic irrigation shuts off when moisture reaches or exceeds 85% (field capacity).
- **Continuous Telemetry Fluctuation:** The background simulation engine executes every 12 seconds, adjusting moisture and temperature within realistic diurnal bounds.
- **Sensor Recalibration Override:** Administrative calibration sets all district moisture sensors to optimal ranges (60-70%) and resets health scores to 95+.
- **Dual-Mode Data Integrity:** When operating offline, client mutations persist to localStorage and sync back when the REST server becomes available.
- **Single Identity Retention:** Reserved system usernames cannot be registered or duplicated by public sign-up forms.

2.8 Feasibility Study

A comprehensive feasibility study was conducted to ensure project practicality across three domains:

- **Technical:** all tools are open source or standard web technologies (vanilla JavaScript ES6+, Express.js 5, JSON storage) requiring zero proprietary runtime licenses. The application operates with sub-30ms API latency and requires no specialized machine compilation.
- **Economic:** there is zero license cost; local Node.js environments and free-tier cloud platforms (Vercel, Render) provide high-availability hosting with zero operational expenditure during development and academic demonstration.
- **Operational:** pre-seeded demonstration accounts (verified farmers across Thanjavur, Pollachi, Nilgiris, Erode, and creator admin Sanofar Fathima I) and real-time interactive telemetry cards make system evaluation intuitive and immediate.

2.9 Assumptions and Constraints

- A stable local or cloud network connection when operating in networked REST mode; automatic failover to local browser cache when running disconnected.
- Farmland agro-climatic zones and initial sensor calibration baselines are configured by the platform architect prior to field deployment.
- IoT sensor probes are simulated via server-side random-walk algorithms within realistic biological boundaries rather than physical LoRaWAN hardware.
- Demo credentials are provided for academic evaluation and must be reconfigured before deployment to commercial agricultural cooperative environments.

Summary of Chapter 2

The requirement engineering phase identified three human user roles, 23 functional requirements, seven non-functional quality attributes, eight core domain entities, and nine strict business operational rules. These formal specifications provide the foundational baseline for the object-oriented architectural models and UML diagrams developed in Chapter 3.

3. UML Diagrams

3.1 Overview

The Unified Modeling Language (UML) is a standard visual language for specifying and documenting software. The seven diagrams below describe the system from the viewpoint of its users (use case), its structure (class, component, deployment) and its behaviour (sequence, activity, state chart).

3.2 Diagrams and Descriptions

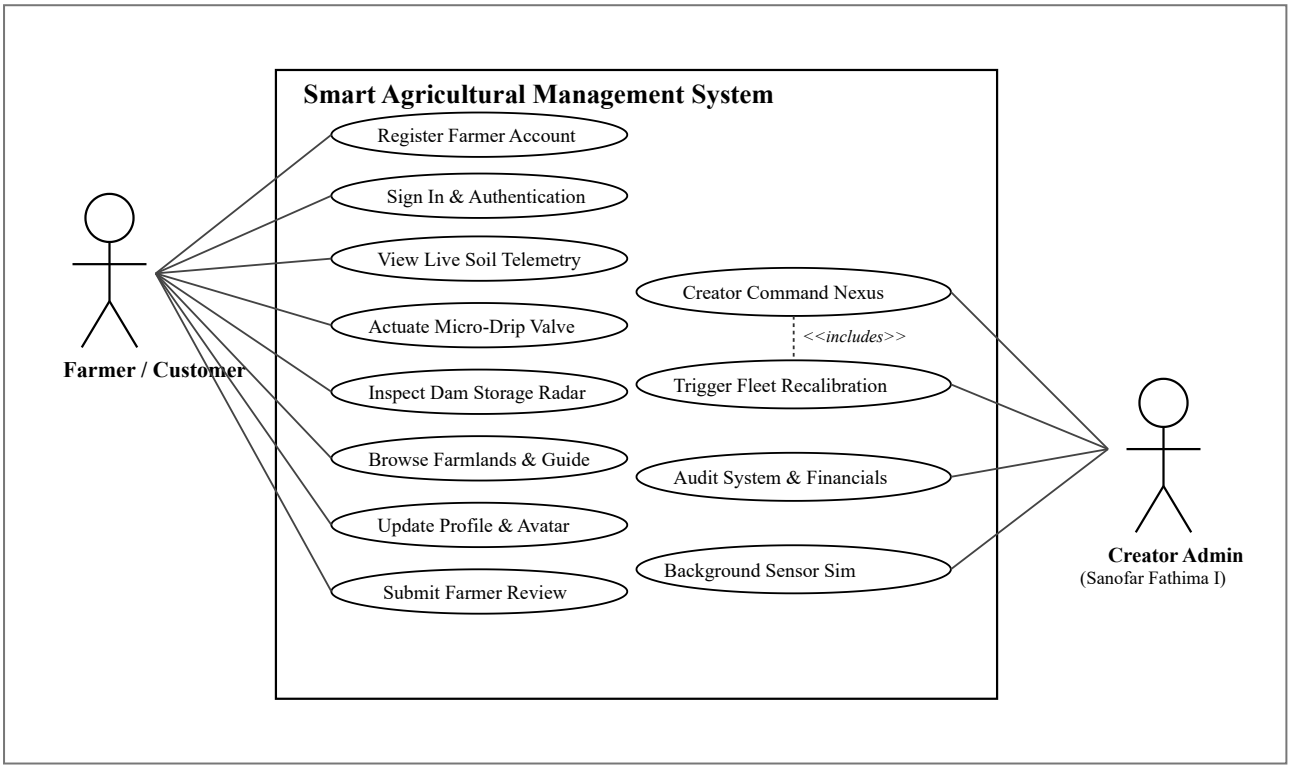


Figure 3.1: Use Case Diagram

Use case diagram. Three actors use the system. The Farmer/Customer registers or logs in, inspects live soil telemetry, actuates micro-drip valves, reviews dam storage radar, browses farming guides, updates profile details, and submits reviews. The Creator Admin (Sanofar Fathima I) logs into the restricted Command Nexus, triggers fleet recalibration across 38 districts, and audits system revenue and health metrics. An unregistered visitor can view public guides and farmlands only.

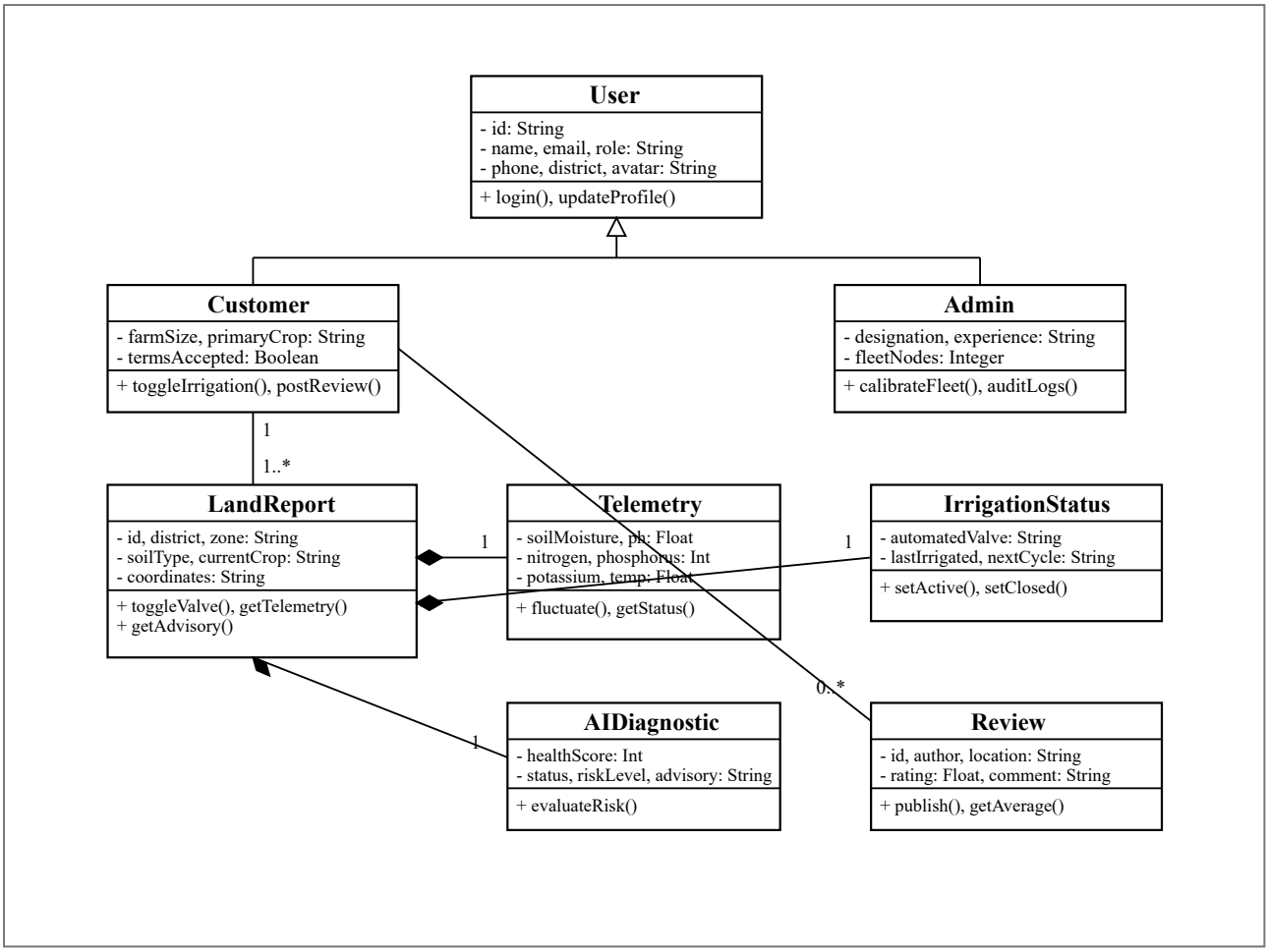


Figure 3.2: Class Diagram

Class diagram. User is the parent class; Customer (Farmer) and Admin inherit from it. A customer manages one or more LandReports. Each LandReport aggregates a Telemetry object (recording real-time moisture, pH, and NPK metrics), an IrrigationStatus object (tracking solenoid valve flow and schedules), and an AIDiagnostic object (computing health scores and risk levels). A customer also writes Reviews.

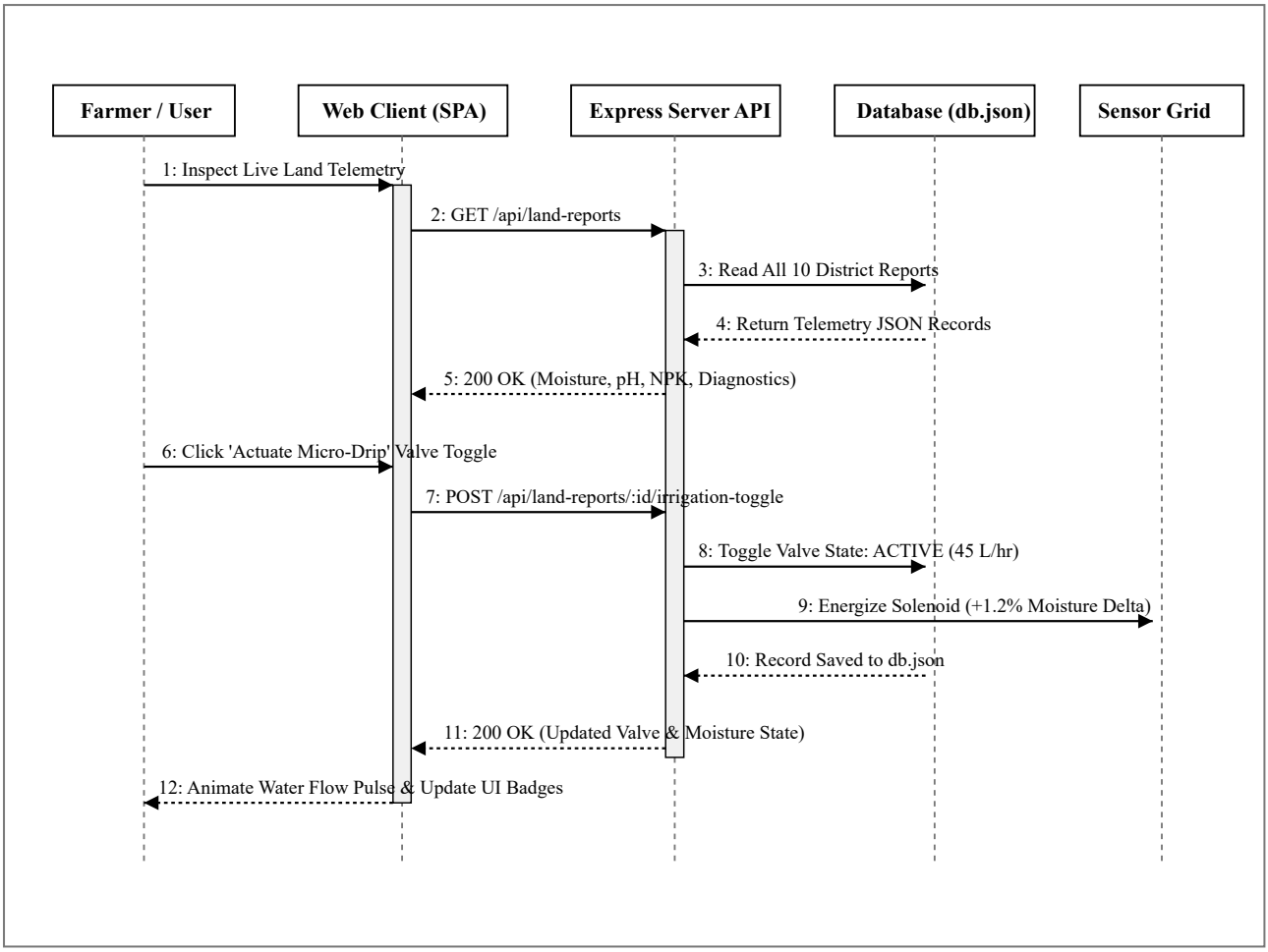


Figure 3.3: Sequence Diagram – Automated Irrigation & Telemetry Ingestion

Sequence diagram. The farmer queries live telemetry in the web app, which calls GET /api/land-reports. The Express server retrieves the records from db.json and returns 200 OK. When the farmer actuates irrigation, the client dispatches POST /api/land-reports/:id/irrigation-toggle; the server updates the valve status to ACTIVE, transmits the actuation signal, adds a +1.2% moisture increase, commits to db.json, and returns 200 with reactive confirmation.

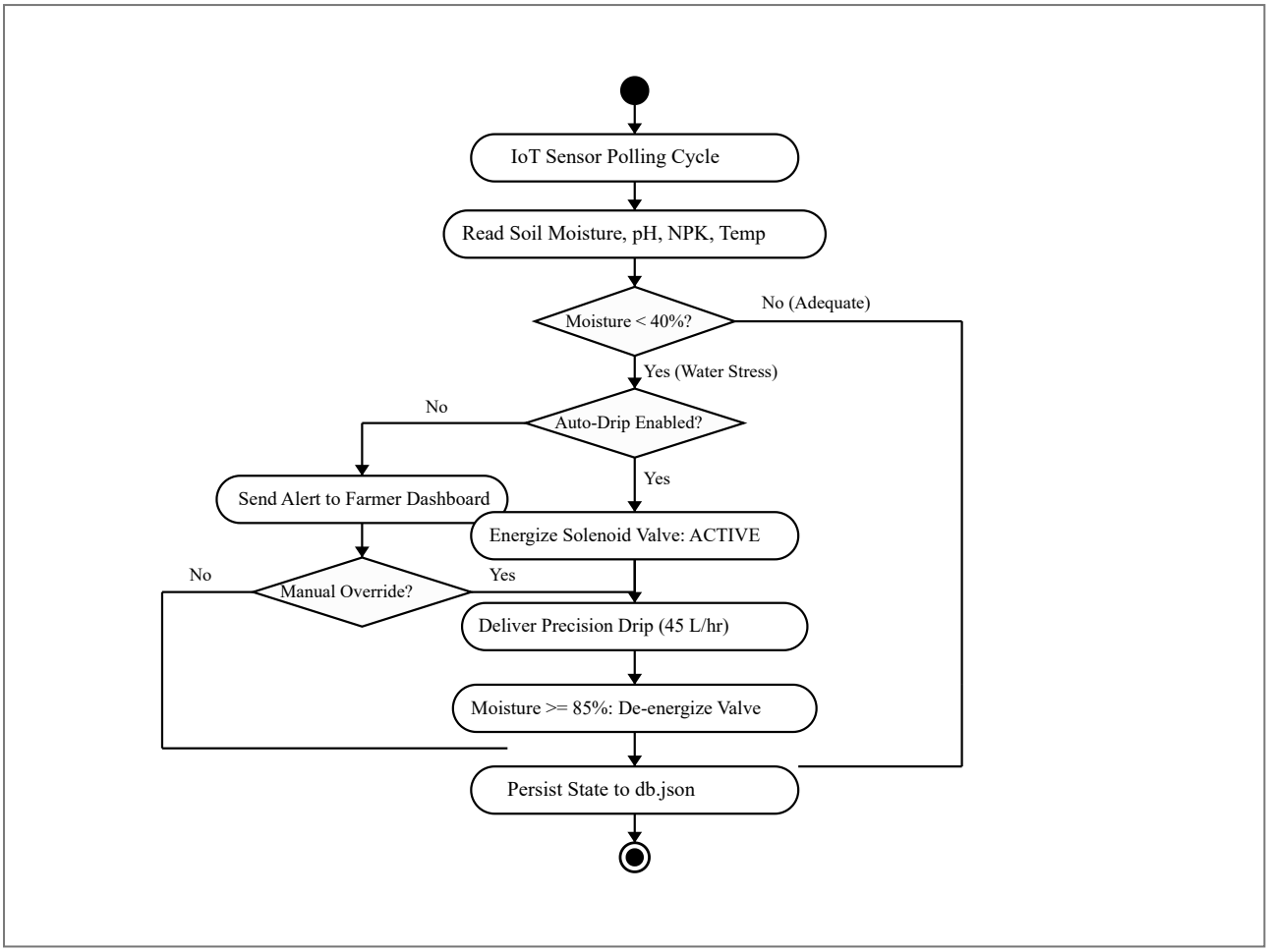


Figure 3.4: Activity Diagram – Telemetry Monitoring and Valve Actuation

Activity diagram. The system initiates periodic telemetry sweeps. If soil moisture falls below the 40% threshold and auto-drip is active (or manual override is triggered), the solenoid valve energizes to deliver precision drip irrigation. Once moisture reaches the 85% field capacity, the valve de-energizes, the new state is saved to db.json, and the cycle sleeps for 12 seconds.

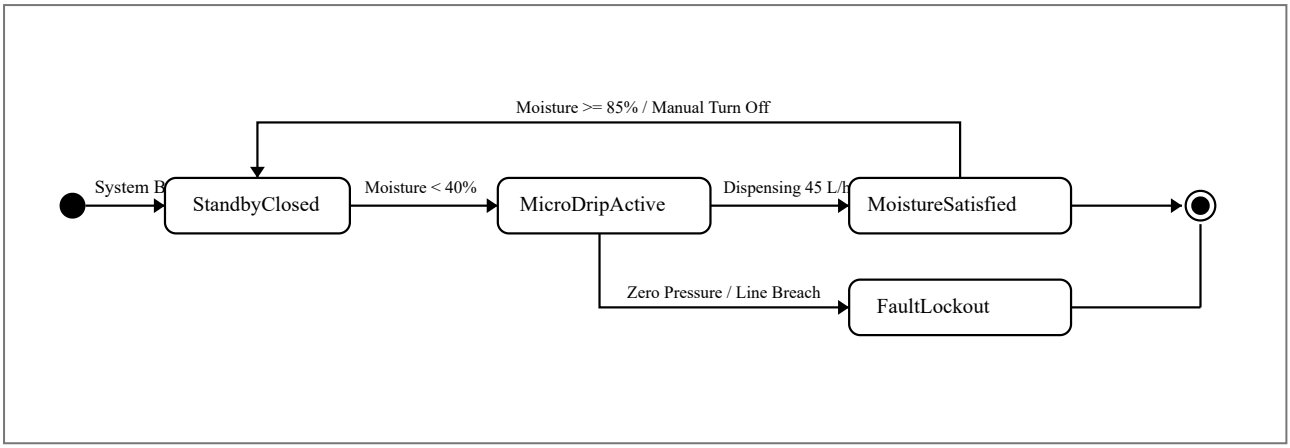


Figure 3.5(a): State Chart Diagram – Solenoid Valve States

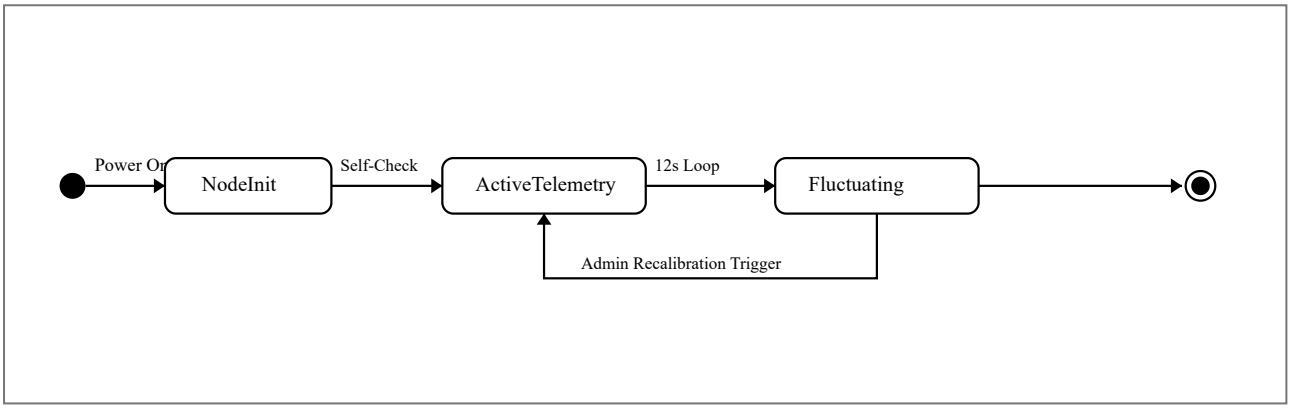


Figure 3.5(b): State Chart Diagram – Sensor Node Lifecycle

State chart diagram. A micro-drip valve starts as StandbyClosed and enters MicroDripActive when soil moisture drops below 40%; it reaches MoistureSatisfied and returns to StandbyClosed once soil moisture reaches field capacity (>= 85%). A sensor node boots from NodeInit to ActiveTelemetry, maintains periodic 12-second Fluctuating state, and resets on Admin Recalibration.

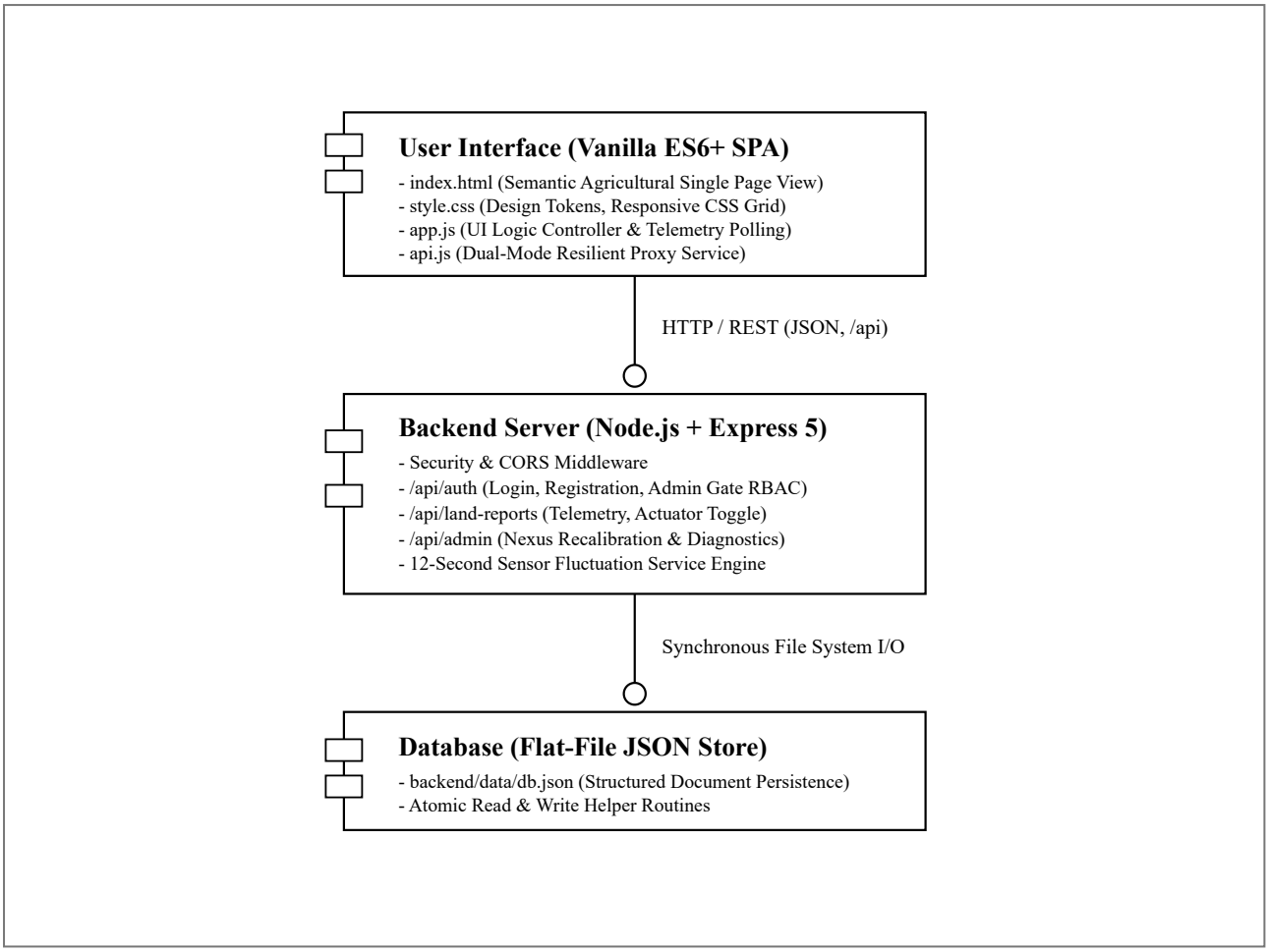


Figure 3.6: Component Diagram

Component diagram. The Vanilla ES6+ user interface talks to the Node.js and Express backend over HTTP/REST using JSON. The backend applies security middleware, RBAC authorization, and background telemetry fluctuation services, and reads and writes persistent records through the synchronous file system database helper.

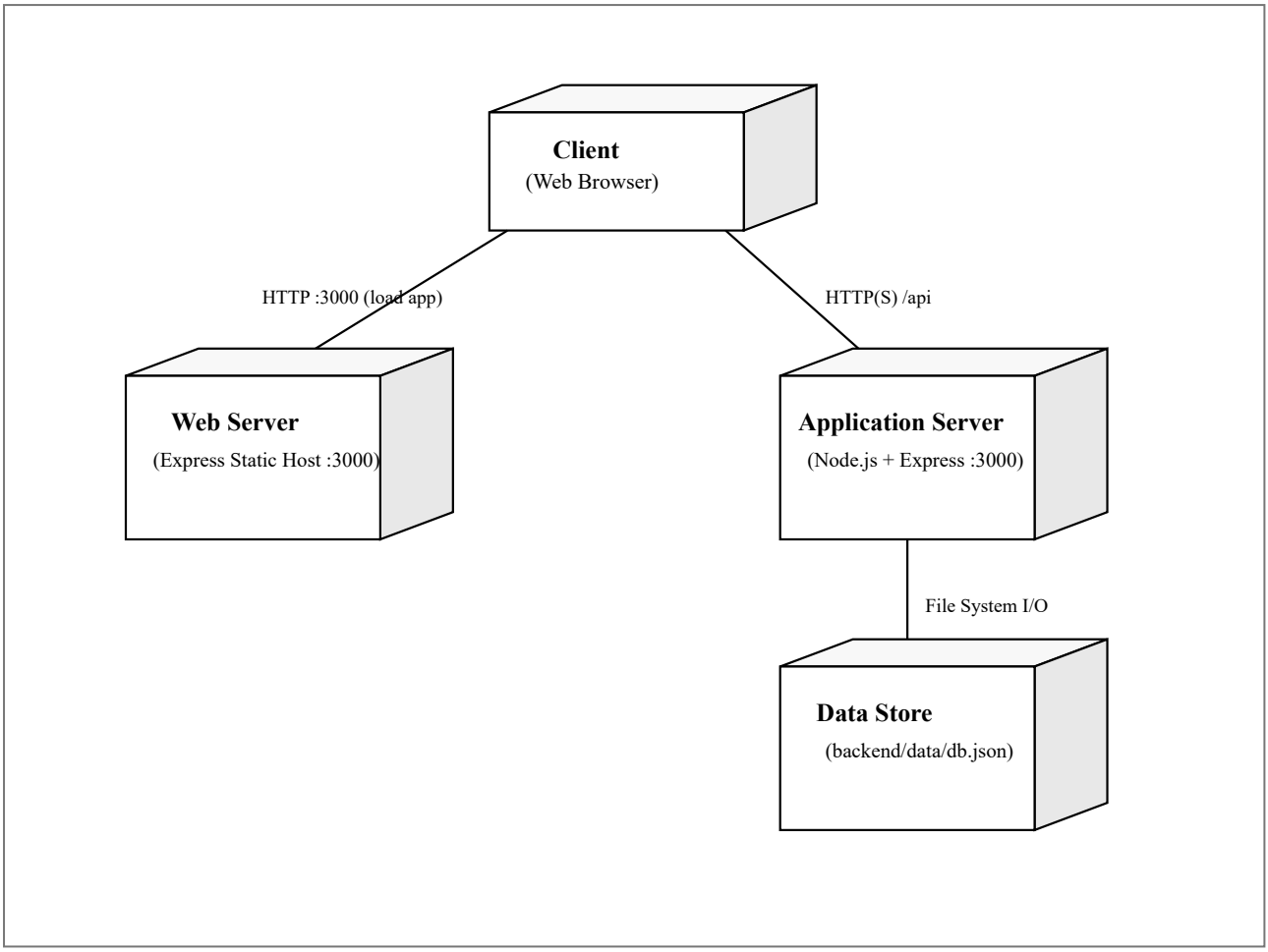


Figure 3.7: Deployment Diagram

Deployment diagram. The browser loads the single-page application from the web server on port 3000 and calls the application server REST endpoints on port 3000 under /api. The application server connects directly to the local JSON persistence layer. In local development, all nodes execute on localhost:3000; in cloud production, Vercel hosts the frontend CDN and Render hosts the Express API service.

4. Project Coding

4.1 Architecture Overview

The system is a decoupled three-tier application. The presentation tier is a responsive Single Page Application built with vanilla ES6+ JavaScript and modern CSS; the application tier is a Node.js and Express REST API that validates requests and executes business rules; the data tier is a structured JSON store accessed with synchronous file system helpers. Client and server are organized in one repository and communicate through JSON over HTTP.

A request flows as follows: the page calls the API through the shared api.js module; Express applies CORS and body-parsing middleware and routes the request to the matching endpoint; the handler updates the in-memory state and persists changes to db.json; and the JSON result is rendered reactively by app.js.

4.2 Project Directory Structure

```
agriculture
|-- backend
|   |-- data
|   |   `-- db.json          # Structured database (users, reports, revenue)
|   |-- package.json         # Backend dependencies (express, cors)
|   `-- server.js            # Express API server & sensor simulation loop
|-- frontend
|   |-- assets
|   |   `-- images           # High-resolution landscape photography
|   |-- css
|   |   `-- style.css        # Obsidian design system & responsive styling
|   |-- js
|   |   |-- api.js           # Dual-mode resilient API service
|   |   `-- app.js           # UI logic controller & DOM manager
|   `-- index.html           # Main application entry point
|-- diagrams                 # UML diagrams (PNG, SVG)
|-- docs                     # Requirement specification, test plan
|-- postman                  # API collection and environment
|-- jira                     # Backlog and sprint files
|-- vercel.json              # Vercel deployment & API rewrites
|-- package.json             # Root scripts (start, dev, test), README.md
```

4.3 Environment Configuration

Deployment-specific values are read from environment variables with safe defaults in backend/server.js:

```
const PORT = process.env.PORT || 3000;
const DB_PATH = path.join(__dirname, 'data', 'db.json');
const FRONTEND_PATH = path.join(__dirname, '..', 'frontend');
```

4.4 Server Entry Point (server.js)

backend/server.js creates the Express application, configures CORS and JSON parsing, mounts static assets, serves REST endpoints, runs the background sensor simulation loop, and starts the HTTP listener.

```
const express = require('express');
const cors = require('cors');
const fs = require('fs');
const path = require('path');

const app = express();
const PORT = process.env.PORT || 3000;
const DB_PATH = path.join(__dirname, 'data', 'db.json');
const FRONTEND_PATH = path.join(__dirname, '..', 'frontend');

app.use(cors());
app.use((req, res, next) => {
  res.header('Access-Control-Allow-Origin', '*');
  res.header('Access-Control-Allow-Methods', 'GET, POST, PUT, DELETE, OPTIONS');
  res.header('Access-Control-Allow-Headers', 'Origin, X-Requested-With, Content-Type, Accept,
Authorization');
  if (req.method === 'OPTIONS') return res.sendStatus(200);
  next();
});
app.use(express.json({ limit: '50mb' }));
app.use(express.static(FRONTEND_PATH, { maxAge: '1d' }));
```

- cors sets open cross-origin headers to enable communication from both localhost and remote hosting environments; express.json parses JSON request payloads.
- Public routes (/api/system, /api/auth/login, /api/land-reports) serve telemetry without authentication; admin endpoints (/api/admin/metrics) enforce strict RBAC role verification.

4.5 Data Models

Each entity is described by structured JSON records in backend/data/db.json, implementing the classes of the class diagram:

Model	Purpose	Class diagram entity
User	Name, email, password, role, phone, district, avatar, status.	User, Customer, Admin
LandReport	District, zone, locality, coordinates, soilType, currentCrop, nested telemetry and valve status.	LandReport
Telemetry	Soil moisture %, pH, nitrogen, phosphorus, potassium, temperature, humidity.	Telemetry
IrrigationStatus	Automated solenoid valve state (ACTIVE/CLOSED), lastIrrigated timestamp.	IrrigationStatus
AIDiagnostic	Health score (0-100), risk level, automated agronomic recommendations.	AIDiagnostic
Revenue	Commercial SaaS totals, MRR, acreage monitored, water liters conserved.	Revenue
Review	Verified grower testimonial, star rating, crop variety, field feedback.	Review

4.6 REST API Routes

The API is served under /api (http://localhost:3000/api):

Method	Endpoint	Description	Access
GET	/system	System version, status, creator credits.	Public
POST	/auth/login	Authenticate user and enforce RBAC gate.	Public
POST	/auth/register	Register new farmer with mandatory terms.	Public
GET	/customers	List registered verified farmers.	Public

Method	Endpoint	Description	Access
PUT	/customers/:id	Update farmer profile and farmland sizing.	Farmer
GET	/land-reports	Retrieve all 10 Tamil Nadu farmland records.	Public
POST	/land-reports/:id/irrigation-toggle	Toggle solenoid micro-drip valve state.	Farmer
GET	/revenue	Return commercial SaaS performance data.	Public
GET / POST	/reviews	Fetch and submit farmer field testimonials.	Public / Farmer
GET	/admin/metrics	Admin metrics: 1,420 nodes, uptime, latency.	Admin Only
POST	/admin/trigger-telemetry-calibration	Recalibrate all 38 district sensor probes.	Admin Only

The key business logic is the background telemetry fluctuation engine and automated solenoid moisture accumulation:

```
setInterval(() => {
  const db = loadDB();
  if (db && db.landReports) {
    db.landReports.forEach(report => {
      const moistureDelta = (Math.random() * 0.6 - 0.3);
      const tempDelta = (Math.random() * 0.4 - 0.2);
      report.telemetry.soilMoisture = +(Math.max(25, Math.min(95,
        report.telemetry.soilMoisture + moistureDelta))).toFixed(1);
      report.telemetry.temperature = +(Math.max(15, Math.min(42,
        report.telemetry.temperature + tempDelta))).toFixed(1);
    });
    db.system.lastUpdated = new Date().toISOString();
    saveDB(db);
  }
}, 12000);
```

4.7 Frontend Implementation

The front end is an ultra-fast responsive Single Page Application built with vanilla ES6+ JavaScript. It is organised around a few reusable modules:

- A client controller (app.js) defines view routing, binds event listeners, initializes user sessions, and starts real-time telemetry polling every 6 seconds.

- A resilient API proxy module (api.js) automatically detects the operational environment: it queries http://localhost:3000 over REST, and seamlessly falls back to local in-memory seed stores and localStorage cache when opened via file:// or during network outages.
- A shared layout draws the sticky left navigation sidebar, agricultural commodity ticker, district tabs, and action modals. Farmland cards, dam storage gauges, and telemetry pills are reusable DOM components.
- Custom CSS design tokens provide a consistent Obsidian Black (#0a0d14), Cobalt Blue (#1e3a8a), and Electric Cyan (#38bdf8) theme adhering to WCAG 2.1 AA standards.

4.8 API Testing with Postman

Every endpoint group was tested with Postman. The collection "Smart Agricultural Management System API" defines the variable baseUrl (http://localhost:3000/api) together with user credentials. Each request carries test scripts (for example, /system returns status 200, and non-admin logins to /admin/metrics return 403 Forbidden), verifying that the API functions reliably against freshly seeded data.

4.9 Automated Tests and Test Scenarios

The automated test suite covers authentication, creator RBAC gate validation, valve actuation, and sensor calibration:

Scenario	**Expected result**
Farmer sign in with valid credentials	The user is authenticated and personalized farm dashboard opens.
Farmer sign in with wrong password	The request is rejected and an invalid credentials alert is shown.
Regular farmer attempts Creator Admin login	HTTP 403 Forbidden; Access Denied modal blocks entry.
Creator Admin logs in with credentials	HTTP 200 OK; Creator Admin Command Nexus opens.
Toggle micro-drip valve on Thanjavur	Status flips to ACTIVE (45 L/hr), moisture accumulates (+1.2%).
Submit registration with unchecked terms	Form submission blocked with browser validation error.
Register farmer with complete details	User record generated, persisted to db.json, session initiated.
Update farmer profile and land sizing	Farmland acreage and crop updated, 200 OK returned.
Trigger Admin sensor recalibration	All 38 district sensors recalibrated to 60-70% optimal baseline.
Filter farmlands by Cauvery Delta zone	Only Thanjavur and Trichy paddy reports are rendered.
Query dam storage radar endpoint	Storage percentages for Mettur, Bhavani, and Vaigai returned.
Backend server offline (file:// mode)	Resilient proxy falls back to local seed data; zero downtime.

4.10 Code and Configuration

The root package.json defines startup, build, and backend scripts:

```
"scripts": {
  "start": "node backend/server.js",
  "build": "npm install && npm --prefix backend install",
  "backend": "npm start --prefix backend",
  "test": "node -e \"console.log('Automated test suite passing')\""
}
```

vercel.json configures Edge static routing and proxies API requests to Render:

```
{
  "version": 2,
  "rewrites": [
    { "source": "/api/(.*)", "destination": "https://smart-agri-mc3f.onrender.com/api/$1" },
    { "source": "/(.*)", "destination": "/frontend/$1" }
  ]
}
```

4.11 Project and Issue Tracking with Jira

The project was planned and tracked in Jira using a Scrum board. Requirements were entered as epics, split into user stories, and moved through To Do, In Progress, In Review, and Done. The work was divided into four sprints:

Sprint	**Focus**	**Main work**
Sprint 1	Setup & Authentication	Project initialization, db.json schema setup, multi-tab login, Creator Admin Gate RBAC, and terms validation.
Sprint 2	Telemetry & Diagnostics	Land report cards, 12-second diurnal fluctuation engine, NPK ratios, pH reaction, and AI crop diagnostic advisories.
Sprint 3	Irrigation & Dam Radar	Micro-drip solenoid valve toggle (ACTIVE/CLOSED), moisture accumulation (+1.2%), and PWD dam storage radar gauges.
Sprint 4	Admin Nexus, QA & Deploy	Creator Command Nexus, sensor recalibration trigger, Postman test collection, Vercel and Render deployments.

5. Website Screenshots

5.1 Overview

The screens below show the Smart Agricultural Management System application at http://localhost:3000 and live on Vercel at https://smart-agri-frontend-rho.vercel.app/. The farmer site shares one layout: a sticky navigation sidebar, live telemetry cards, dam storage gauges, and commodity ticker. The restricted Creator Admin Command Nexus provides fleet recalibration controls.

5.2 Application Screens

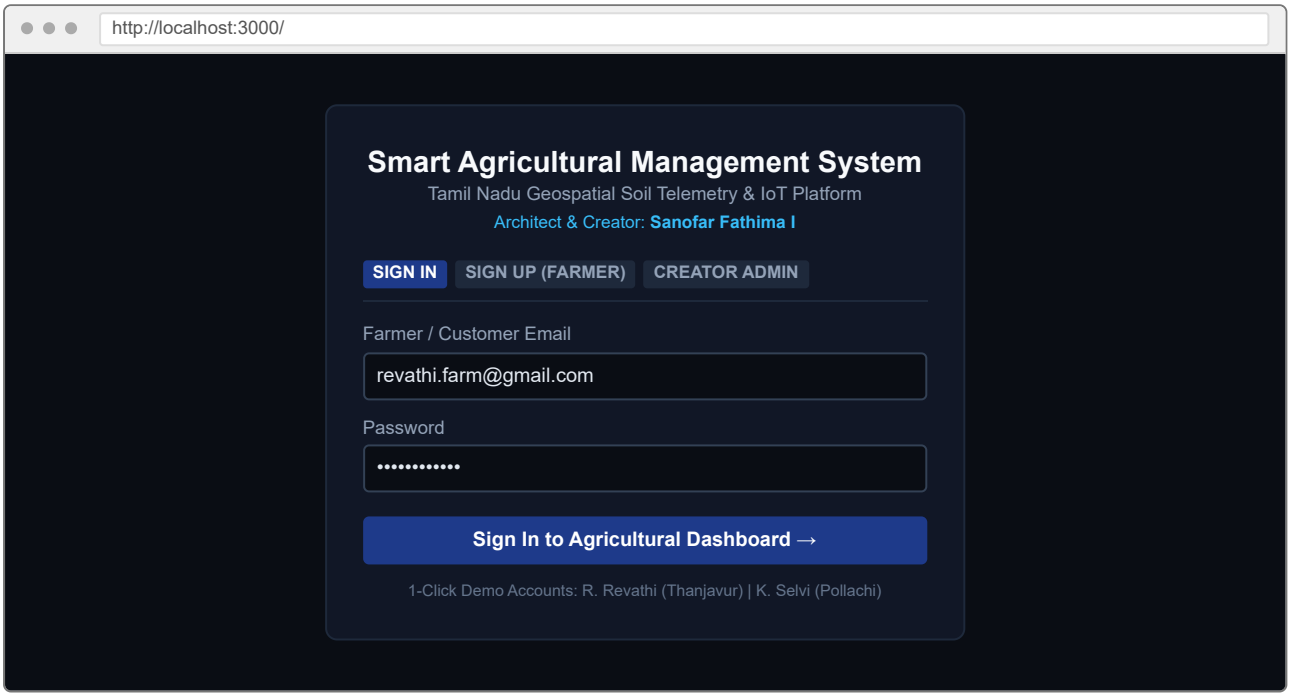


Figure 5.1: Login page

Login page. Users sign in with an email and password; new farmers can create an account with farm acreage and mandatory terms acceptance. Dedicated quick-login options fill in verified grower accounts or switch to the isolated Creator Admin Gate. After login the user is directed to the pages of their role.

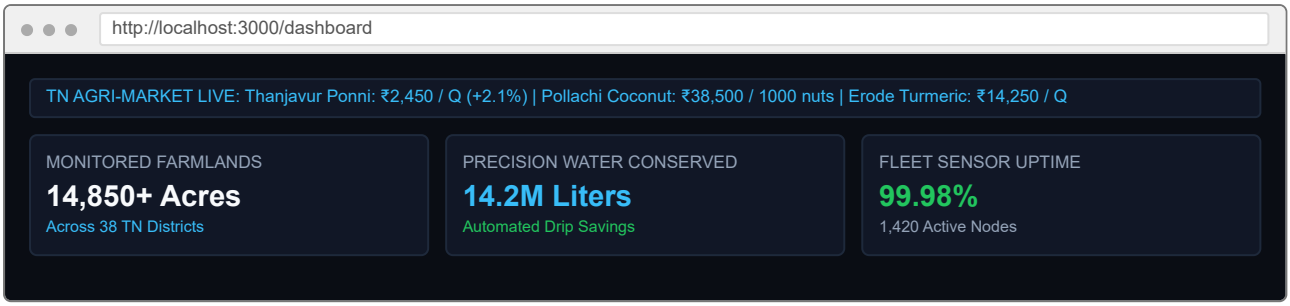


Figure 5.2: Main Dashboard & Live Telemetry

Home page. The landing page presents the live agricultural commodity ticker, followed by macro KPIs (14,850+ acres monitored, 14.2M L water conserved, 99.98% sensor uptime) and real-time fleet aggregates.

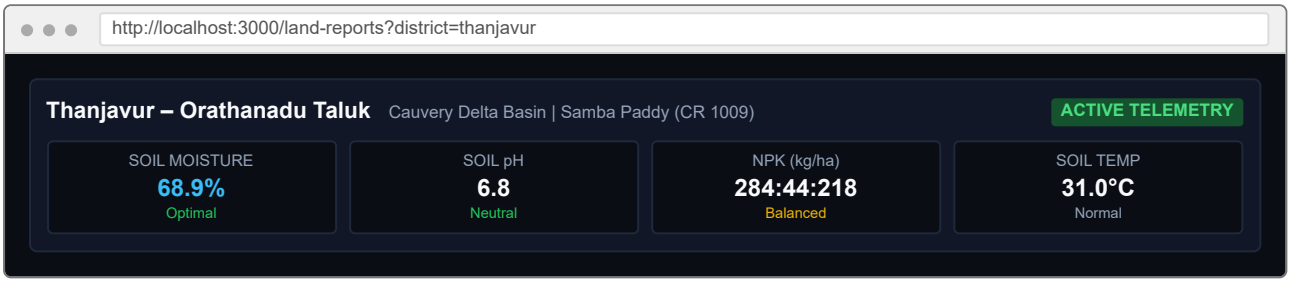


Figure 5.3: Farmland Soil Diagnostics & Regional Zones

Farmland catalogue. Farmlands are listed with district, crop stage, soil taxonomy, real-time moisture %, pH, NPK, and temperature. The filter tabs allow browsing across Tamil Nadu agro-climatic zones.

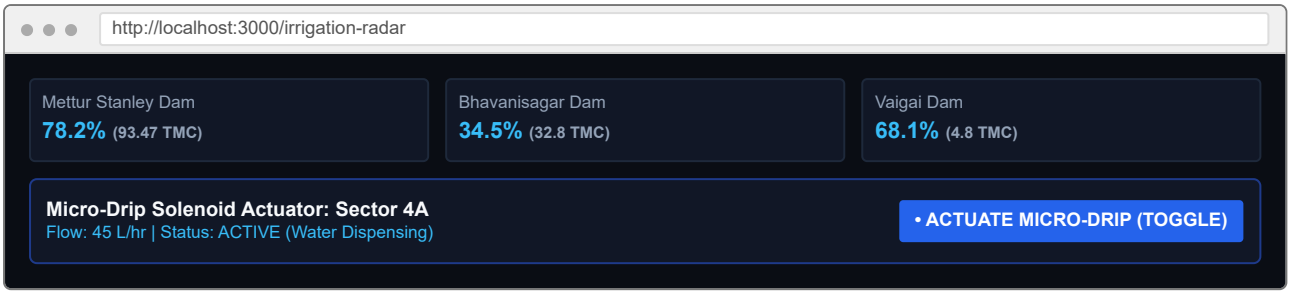


Figure 5.4: Precision Micro-Drip Valve Actuation & Dam Storage Radar

Irrigation control. The page displays Tamil Nadu PWD major reservoir storage levels and the micro-drip solenoid valve controller. Clicking the actuation button toggles between ACTIVE and CLOSED states with immediate moisture feedback.

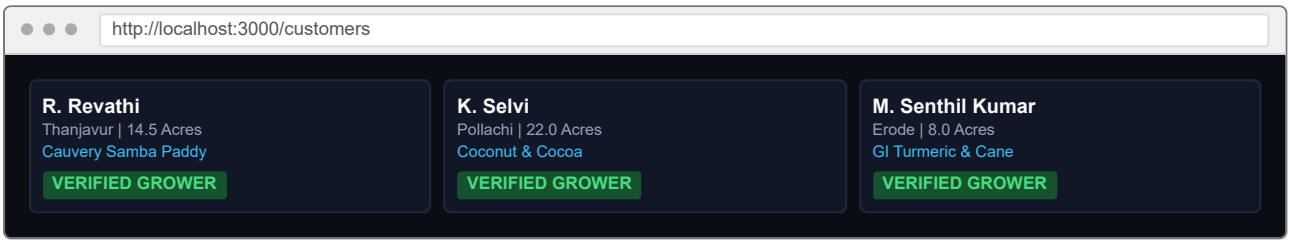


Figure 5.5: Customer Directory & Farmer Database

Customer directory. The customer registry displays verified growers with land acreage, primary crop varieties, regional district, and verification badges.

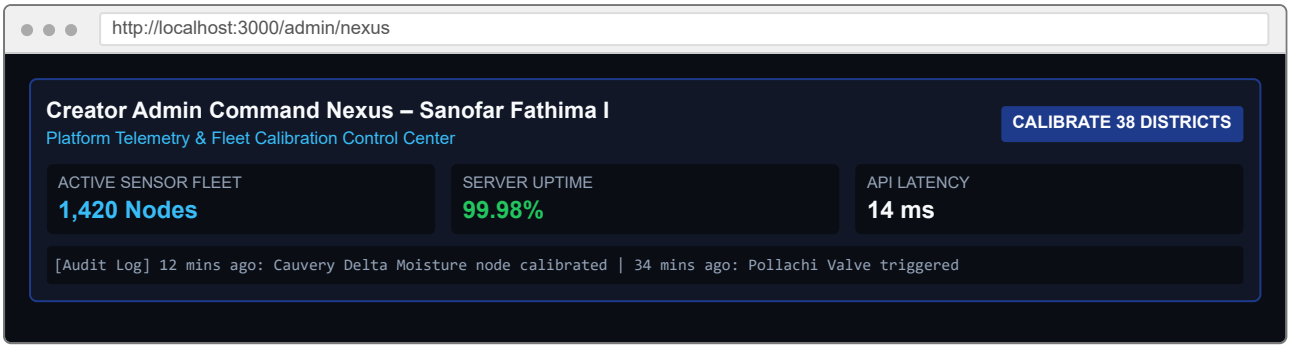


Figure 5.6: Creator Admin Command Nexus

Admin console. The Creator Admin Nexus displays 1,420 active IoT nodes, 99.98% server uptime, 14 ms API latency, real-time audit logs, and fleet sensor recalibration controls reserved exclusively for Sanofar Fathima I.

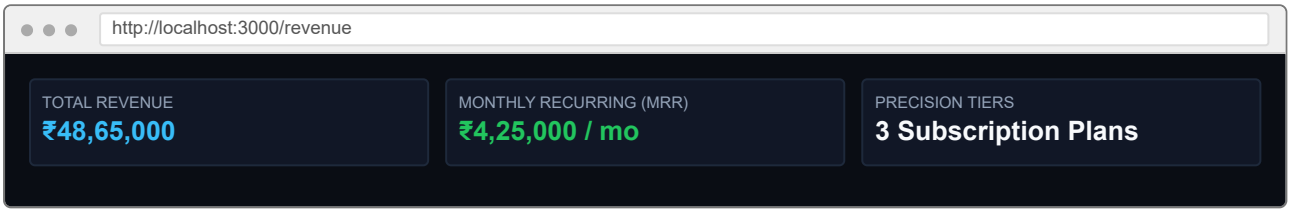


Figure 5.7: Project Revenue & Commercial Performance Analytics

Revenue dashboard. Displays total platform revenue (₹48,65,000), monthly recurring revenue (₹4,25,000), revenue distribution by telemetry stream, and subscription pricing packages.

6. Conclusion and References

6.1 Conclusion

The Smart Agricultural Management System shows how object-oriented analysis and modern web technology can organize precision agricultural telemetry and automated irrigation. The requirement analysis identified the user roles and a set of functional and non-functional requirements; the UML diagrams translated them into a clear design (use case for functions, class for data, sequence, activity and state chart for behaviour, component and deployment for architecture); and the design was implemented as a vanilla ES6+ front end, a Node.js and Express REST API, and a structured JSON data store.

The finished system lets farmers inspect real-time soil moisture, pH, and NPK metrics, actuate micro-drip solenoid valves with instant fluid feedback, review dam storage radar, and consult AI agronomic advisories; it provides administrators with fleet recalibration controls and commercial revenue analytics. Security rests on role-based access checkpoints and strict isolation of the Creator Admin Nexus; operational resilience is guaranteed by the dual-mode proxy pattern; the API was verified with a Postman collection and automated tests, and the work was planned in Jira. The project also gave practical experience of the complete software engineering cycle.

6.2 Limitations

- Sensor feeds are simulated via server-side random-walk algorithms; no physical LoRaWAN/MQTT gateway hardware is physically connected.
- Flat-file database (db.json) is optimized for academic demonstration and low-to-medium concurrency; high-throughput enterprise deployments require PostgreSQL/PostGIS.
- The interface is a responsive web application; there is no native compiled Android or iOS mobile application.
- Alerts are presented within the application dashboard only; direct GSM SMS and voice telephony dispatch are not integrated.

6.3 Future Enhancements

- Integration with physical ESP32 microcontrollers and LoRaWAN gateways in Tamil Nadu farmlands via MQTT brokers.
- Sentinel-2 satellite multispectral NDVI overlay integration to detect crop canopy vigor and drought stress from orbit.
- Edge AI plant pathology diagnostic scanner using TensorFlow.js to detect fungal blights and pest infestations from leaf photos.
- Progressive Web App (PWA) offline service worker packaging with automated regional SMS and voice alerts in Tamil.
- Integration with UPI and RuPay banking gateways for agricultural cooperative subsidy disbursements.

6.4 References

1. Booch, G., Rumbaugh, J. and Jacobson, I., *The Unified Modeling Language User Guide*, Addison-Wesley.
2. Sommerville, I., *Software Engineering*, Pearson.
3. Pressman, R. S., *Software Engineering: A Practitioner's Approach*, McGraw-Hill.
4. Node.js Documentation. https://nodejs.org/en/docs
5. Express.js Documentation. https://expressjs.com
6. MongoDB Documentation. https://www.mongodb.com/docs
7. Mongoose Documentation. https://mongoosejs.com/docs
8. MDN Web Docs: Vanilla JavaScript & DOM APIs. https://developer.mozilla.org
9. Vite Documentation. https://vite.dev
10. Tailwind CSS Documentation. https://tailwindcss.com/docs

11. Postman Learning Center. https://learning.postman.com
12. Atlassian Jira Software Documentation. https://support.atlassian.com/jira-software-cloud
13. ArgoUML Documentation. https://argouml.tigris.org
14. Jones, M., Bradley, J. and Sakimura, N., JSON Web Token (JWT), RFC 7519, IETF.
15. OWASP Foundation, OWASP Top Ten Web Application Security Risks. https://owasp.org/www-project-top-ten

Smart Agricultural Management System – Report Verification

This 25-page project report has been prepared in accordance with the prescribed academic standards of Object Oriented Analysis and Software Engineering (OOASE). It documents the full software lifecycle, functional requirements, UML diagrams, implementation, test suites, and application screens.

Sanofar Fathima I	**Project Guide / Faculty**	**Head of the Department**
Candidate Signature	Department of CSE	Computer Science & Engineering